Technical Assessment: Mid-Level Android Developer

1. OBJECTIVE

Build a master-detail gallery application named **"The Breed Explorer App"** using 100% Jetpack Compose. Traditional XML view hierarchies or layout files are not allowed.

> **Note on UI:** You are entirely responsible for how the app looks and is laid out. However, focus on your architecture, state management, and code choices rather than spending hours trying to make the design pixel-perfect.

2. CORE FUNCTIONAL REQUIREMENTS

- **Network Layer:** Fetch the complete list of all dog breeds from the master directory endpoint.
- **Dual-Screen Navigation:**
  - *Screen 1 (Breeds List):* Display the list of dog breeds in a scrollable view. Every breed entry must be clickable to open the next screen.
  - *Screen 2 (Gallery Grid):* When a breed is clicked, pass that breed to the second screen and fetch a random batch of up to 20 images specifically for that breed. Display these images in a grid layout. Include a working back button to return to the list screen.
- **State Management:** Properly handle async data streams and loading states. The UI needs to cleanly show loading indicators when fetching data, empty states when there's no content, and a graceful fallback/retry option if a network request fails.
- **Image Loading:** Load the network images efficiently, handling placeholders and loading failures properly.

3. API SPECIFICATIONS

Endpoint 1 (Master Breed Directory):

```
GET https://dog.ceo/api/breeds/list/all
```

Endpoint 2 (Dynamic Breed Gallery):

```
GET https://dog.ceo/api/breed/{breed_name}/images/random/20
```

4. TECHNICAL GUARDRAILS

- **Architecture:** Use a standard architectural pattern to cleanly separate your UI layer from your data models and business logic.
- **Dependency Injection & Concurrency:** Use standard tools to manage your dependencies and make sure all network operations are safely offloaded from the main UI thread.

5. BONUS CRITERIA

- **Favorites UI State:** Add a feature to allow users to toggle a "Favorite" status on images within the gallery grid. This state only needs to live in memory during the app's current lifecycle.
- **Interactivity:** Add a search bar to filter the breeds list, or add a pull-to-refresh mechanic on the gallery grid.

6. EVALUATION MATRIX

Dimension	What We Look For
State Architecture	How you handle async operations, retain data through screen rotations, and expose data streams to the UI.
Data Lifecycle	Clean mapping of API payloads into your own models and proper isolation of the network layer.
Code Cleanliness	Decoupling of components, how navigation is structured, and how you scope dependencies.